

R-Control IP4 – HTTP JSON REST API ("/api/... ")

This document describes the HTTP/HTTPS JSON REST API of the R-Control IP4. All information is derived from the firmware implementation and refers to the endpoints currently included in this project.

1. Transport, Base URL, and Ports

- **Base URL:** `http(s)://<device>:<port>`
- **HTTP port:** `config.network.lan.eth0.http.port` (default: 80)
- **HTTPS** (optional): `config.network.lan.eth0.https.enabled`
 - Certificate/key are loaded from SPIFFS:
 - `config.network.lan.eth0.https.cert_path` (default: `/spiffs/server.crt`)
 - `config.network.lan.eth0.https.key_path` (default: `/spiffs/server.key`)
 - If HTTPS is enabled but certificate/key are not readable, the device automatically falls back to HTTP.

Note: The firmware supports both HTTP and HTTPS depending on build configuration (ESP-IDF HTTPS server feature) and runtime configuration.

2. Authentication

2.1 Bearer Token

All endpoints under `/api/...` (except login) require a valid bearer token:

Authorization: `Bearer <token>`

If the header is missing/too long or the token is invalid:

- **HTTP 401 Unauthorized**
- Header: `WWW-Authenticate: Bearer realm="R-Control-IP4"`
- Body:

```
{"error": "unauthorized"}
```

2.2 Login – `POST /api/auth/login`

Request

- Method: `POST`
- Path: `/api/auth/login`
- Body: JSON (max. 1024 bytes)

```
{  
  "username": "admin",  
  "password": "<password>",  
}
```

```
"ttl": 3600
}
```

- `ttl` is optional and is clamped to **60...86400 seconds**.

Response (200 OK)

```
{
  "token": "<jwt>",
  "token_type": "Bearer",
  "expires_in": 3600,
  "user": {
    "name": "admin",
    "username": "admin",
    "permissions": ["settings", "overview", "about"]
  }
}
```

Error cases

- **401 Unauthorized** with `{"error": "invalid_credentials"}` (wrong credentials)
- **400 Bad Request** (invalid length/JSON; the firmware may also return text error responses here)

Example (curl)

```
curl -sS -X POST "http://r-control-ip4.local/api/auth/login" \
-H "Content-Type: application/json" \
-d '{"username": "admin", "password": "admin123", "ttl": 3600}'
```

3. Permissions

Some endpoints require, in addition to authentication, a permission in the user profile:

- **settings**: configuration, user management, restart/OTA, factory provisioning
- **overview**: telemetry/overview (temperature history, IO log)
- **about**: read firmware/factory information (partly as an alternative to **settings**)

Important: Multiple endpoints check **authentication only** (no permission). This is documented deliberately because it reflects the current behavior.

4. Response Formats and Error Handling

4.1 Success Responses

- Success responses are typically JSON.
- Many responses set cache-busting (**Cache-Control: no-store, no-cache, must-revalidate**).

4.2 Standard JSON Errors

A large portion of validation/permission-related errors uses:

```
{"error": "<code>", "message": "<description>"}
```

Examples:

- 403 Forbidden: {"error": "forbidden", "message": "Permission 'settings' required"}
- 400 Bad Request: {"error": "invalid_json", "message": "Invalid JSON"}

4.3 Text Error Responses

Some handlers use ESP-IDF `httpd_resp_send_err(...)` and return **text** (not JSON). Clients should therefore be robust and handle both JSON and text errors.

5. Endpoints

5.1 Network

GET /api/network

- Auth: required
- Permission: none (auth-only)
- Response: `config.network` as JSON.

GET /api/network/status

- Auth: required
- Permission: none (auth-only)
- Response: live status (example structure; fields may be empty):

```
{
  "enabled": true,
  "protocol": "dhcp+fallback",
  "netifReady": true,
  "ip": "192.168.0.3",
  "netmask": "255.255.255.0",
  "gateway": "192.168.0.1",
  "ipAssigned": true,
  "mac": "AA:BB:CC:DD:EE:FF",
  "linkUp": true,
  "online": true,
  "dhcpActive": true,
  "dhcpStatus": "started",
  "dns": ["192.168.0.1"],
  "source": "dhcp",
  "internetReachable": true,
  "wanIpValid": true,
  "wanFetching": false,
  "wanCheckedMs": 0,
```

```
"wanIp": "203.0.113.10",  
"wanGeoValid": false  
}
```

POST /api/network/wan-refresh

- Auth: required
- Permission: none (auth-only)
- Starts a WAN/Geo refresh in the background.
- Response:
 - {"status": "started"} or {"status": "busy"}

POST /api/network/geo/address

- Auth: required
- Permission: settings
- Body: JSON (max. 1024 bytes)
 - Required: street, houseNumber, city, postalCode
 - Optional: country
- Response: status: ok + geo data.
- Typical errors:
 - 400 missing_fields
 - 404 geo_not_found
 - 400 geo_invalid
 - 502 geo_failed

POST /api/network/eth0

- Auth: required
- Permission: none (auth-only)
- Body: JSON (max. 2048 bytes) – allowed string keys:
 - protocol, ip, netmask, gateway, dns, hostname
- Response:

```
{"status": "updated"}
```

POST /api/network/apply

- Auth: required
- Permission: none (auth-only)
- Body: none
- Response:

```
{"status": "restarting"}
```

- Behavior: restart is triggered with a delay after the response.

5.2 System

GET /api/system

- Auth: required
- Permission: none (auth-only)
- Response: `config.system` as JSON.

POST /api/system/restart

- Auth: required
- Permission: `settings`
- Response: `{"status": "restarting"}`
- Behavior: restart is triggered with a delay after the response.

POST /api/system/ota

- Auth: required
- Permission: `settings`
- Body: **binary firmware image** (not JSON)
- Limits:
 - Payload must not be empty
 - Payload must fit into the OTA partition (otherwise `413 image_too_large`)
- Response (example):

```
{
  "status": "ok",
  "written": 1048576,
  "partition": "ota_0",
  "reboot": true,
  "firmware": {
    "version": "1.2.3",
    "project": "R-Control-IP4",
    "build_date": "Jan 01 2026",
    "build_time": "12:34:56"
  }
}
```

- Behavior: restart is triggered with a delay after the response.

5.3 Online Firmware Update (GitHub)

These endpoints use `config.system.firmwareUpdate.*` (e.g. `enabled`, `owner`, `repo`, `asset`).

GET /api/system/fw-update/check

- Auth: required

- Permission: `settings`
- Response (example):

```
{
  "status": "ok",
  "currentVersion": "1.0.0",
  "latestVersion": "1.1.0",
  "updateAvailable": true,
  "assetName": "R-Control-IP4.bin",
  "assetUrl": "https://...",
  "prerelease": false,
  "draft": false
}
```

POST /api/system/fw-update/install

- Auth: required
- Permission: `settings`
- Starts the online update in the background.
- Response: `202 Accepted` with `{"status": "started", ...}`
- Errors: `409 fw_update_busy`, `400 fw_update_disabled`, `400 fw_update_not_configured`, `502 fw_update_check_failed`

GET /api/system/fw-update/status

- Auth: required
- Permission: none (auth-only)
- Response:

```
{"status": "ok", "state": "idle", "message": "...", "error": ""}
```

5.4 Scheduler

GET /api/scheduler

- Auth: required
- Permission: none (auth-only)
- Response: scheduler configuration (JSON), plus:
 - `runtime_rule_count`
 - `runtime_max_rules`

POST /api/scheduler

- Auth: required
- Permission: none (auth-only)
- Body: JSON object (max. 64 KiB)

- Special case:
 - If the payload contains `astro` or `location`, these data are persisted to `config.system.astro`.
- Response: `status: ok|error` and `data` (current scheduler config with runtime metadata).

5.5 Actions

GET /api/actions

- Auth: required
- Permission: none (auth-only)
- Response: actions configuration (JSON) + runtime metadata analogous to scheduler.

POST /api/actions

- Auth: required
- Permission: none (auth-only)
- Body: JSON object (max. 64 KiB)
- Response: `status: ok|error` and `data` (current actions config with runtime metadata).

5.6 Configuration

GET /api/config

- Auth: required
- Permission: none (auth-only)
- Response: complete active configuration as JSON (streaming), **without** `config.auth.jwt_secret`.

POST /api/config

- Auth: required
- Permission: none (auth-only)
- Body: JSON (max. 64 KiB)
- Behavior:
 - If `config.auth.jwt_secret` is already set, it is preserved when replacing the configuration.
- Response: `{"status": "ok"}`

GET /api/config/bundle

- Auth: required
- Permission: `settings`
- Response: `application/x-tar` download containing:
 - `config.json`
 - `scheduler.json`
 - `actions.json`

Important note: This bundle contains the current configuration, including sensitive fields (e.g. `jwt_secret`). Treat the export like a password backup.

POST /api/config/bundle

- Auth: required
- Permission: `settings`
- Body: `application/x-tar` upload (max. approx. 768 KiB)
- Expected contents: `config.json`, `scheduler.json`, `actions.json`
- Response: `{"status":"imported","reboot":true}`
- Behavior: restart is triggered with a delay after the response.

POST `/api/config/restore-defaults`

- Auth: required
- Permission: `settings`
- Response: `{"status":"restored"}`
- Behavior: factory defaults are loaded and the device restarts afterwards.

5.7 Telemetry / Overview

GET `/api/telemetry/temperature`

- Auth: required
- Permission: `overview`
- Response: streaming JSON
 - `intervalSeconds`
 - `capacity`
 - `minIntervalSeconds`, `maxIntervalSeconds`
 - `samples`: array of `{ "t": <timestampMs>, "v": <temperatureC> }`

GET `/api/overview/actions`

- Auth: required
- Permission: `overview`
- Response: `entries` array of IO/relay events including plain-text fields.

5.8 About

GET `/api/about/app`

- Auth: required
- Permission: `settings` **or** `about`
- Response:

```
{"status":"ok","appVersion":"...","idfVersion":"..."}
```

5.9 Factory (Production)

These endpoints are intended for provisioning/production.

GET `/api/factory/info`

- Auth: required
- Permission: **settings** or **about**
- Response contains, among others, **articleNumber**, **serialNumber**, **productionDate**, **macAddress**, **factoryMac**, **locked**, **ntcCalibration**.

POST /api/factory/provision

- Auth: required
- Permission: **settings**
- Body: JSON (max. 1024 bytes)
 - Required: **articleNumber**, **serialNumber**, **productionDate** (YYYY-MM-DD)
 - Optional: **macAddress** (sets base MAC), **hostname**
- Existing data may be **overwritten**. The only write protection is **POST /api/factory/lock**; an overwrite is logged as a warning.
- Errors:
 - **409 factory_locked** — factory area is locked
 - **400 missing_fields**, **400 invalid_date**, **400 invalid_mac**, **400 invalid_chars**, **400 invalid_hostname**

NTC calibration is **not** handled by this endpoint — use **POST /api/factory/ntc-calibrate**.

POST /api/factory/ntc-calibrate

- Auth: required
- Permission: **settings**
- Body: JSON (max. 256 bytes), **exactly one** of:
 - **refTempC** (number) — reference resistor point, only -25, 25 or 55 (± 2 °C)
 - **fault** (string) — **"open"** (nothing connected) or **"short"** (input bridged)
- Independent of provisioning and **locked**; re-calibration is always possible.
- Errors:
 - **400 invalid_request** — neither or both fields set
 - **400 invalid_fault** — **fault** is neither **"open"** nor **"short"**
 - **400 ntc_ref_temp_unsupported** — **refTempC** not near -25/25/55
 - **400 ntc_measurement_implausible** — reading does not match the expected point
 - **409 ntc_no_measurement** — no valid reading yet (wait ~3 s after rewiring)

NTC calibration procedure

Five calls, one per measurement point:

Step	At the input	Body
1	86430 Ω	{"refTempC": -25}
2	10000 Ω	{"refTempC": 25}
3	3536 Ω	{"refTempC": 55}
4	nothing connected	{"fault": "open"}

Step	At the input	Body
5	input bridged	<code>{"fault": "short"}</code>

Do steps 1–3 first: the fault anchors are sanity-checked against p1/p3.

Steps 4 and 5 need no reference resistors. They provide the "not connected" and "short circuit" thresholds as a **direct measurement** instead of extrapolating them from the divider model — that extrapolation was sensitive enough that a 1% error in steps 1–3 could push the open threshold past the real reading and silently disable open-circuit detection.

Wait at least one sample period (2 s, better 3 s) between steps: calibration uses the most recently measured mV value.

Example:

```
curl -X POST http://DEVICE/api/factory/ntc-calibrate \
  -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' \
  -d '{"fault": "open"}'
```

The `ntcCalibration` object (returned by `ntc-calibrate`, `provision` and `GET /api/factory/info`) contains: `offsetC`, `rScale`, `vddMv`, `fixedOhm`, `dividerCalibrated`, `mvCorrectionCalibrated`, `p1Mv`, `p2Mv`, `p3Mv`, `openMv`, `shortMv`, `faultAnchorsCalibrated`, `openThresholdMv`, `shortThresholdMv`, `isDefault`.

POST /api/factory/lock

- Auth: required
- Permission: `settings`
- Locks the factory area (once provisioning is complete).

5.10 User Management

All user endpoints require `settings`.

GET /api/users

- Response:

```
{"limit": 8, "users": [ ... ]}
```

POST /api/users

- Body: JSON (max. 4096 bytes)
 - Required: `username`, `password`
 - Optional: `permissions` (array)
- Response: `201 Created` with `{"status": "created"}`

PUT /api/users/<username>

- Body: JSON (max. 4096 bytes)
 - Optional: `permissions` (array)
 - Optional: `password` (string, additionally requires `currentPassword`)
 - Optional: `currentPassword`
- Response: `{"status":"updated"}`

DELETE /api/users/<username>

- Response: `{"status":"deleted"}`
- Error: `409 last_user` (the last user cannot be removed)

6. Best Practices & Security

- Use **HTTPS** as soon as the device is operated outside a fully trusted network.
- Treat tokens and configuration exports (especially `/api/config/bundle`) as **secret**.
- Write/reset/OTA functions should only be granted to accounts with `settings`.
- Note that some endpoints currently check authentication only (no permission). If you need a more restrictive model, this must be implemented in the firmware.

7. Troubleshooting

- `401 Unauthorized`: token missing/invalid/expired or `Authorization` header too long.
- `403 Forbidden`: user does not have the required permission.
- `400 invalid_json`: JSON is syntactically invalid.
- After `.../apply`, `.../restore-defaults`, `.../ota` or bundle import, a **restart** is part of the normal flow.